VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELGAUM-590 014



2023-2024

A Mini Project Report on
"DIGITAL IMAGE FORGERY DETECTION"

Submitted in partial fulfilment for the award of degree of

Bachelor of Engineering in

Computer Science & Engineering

by

RISHABH KUMAR 1AT21CS085

ROHITH H CHAVAN 1AT21CS086

SANSKAR RAJ 1AT21CS097

MUTHURAJ R 1AT22CS407

Under the Guidance of

Prof. KEERTHI KS

Asst. Professor, Dept. of CS&E, AIT

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



ATRIA INSTITUTE OF TECHNOLOGY

BANGALORE - 560024

2023-2024

ATRIA INSTITUTE OF TECHNOLOGY

Department of Compute Science & Engineering
BANGALORE - 560024



CERTIFICATE

Certified that the project work entitled **"DIGITAL IMAGE FORGERY DETECTION"**, carried out by **RISHABH KUMAR (1AT21CS085), ROHITH H CHAVAN (1AT21CS086), SANSKAR RAJ (1AT21CS097), MUTHURAJ R (1AT22CS407),** a bona fide students of **Atria Institute of Technology,** in partial fulfilment for the award of Bachelor of Engineering in **Computer Science & Engineering of Visvesvaraya Technological University, Belgaum** during the academic year 2023-2024. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The project report has been approved as it satisfies requirement in respect of project work prescribed for the said degree.

DECLARATION

WE, RISHABH KUMAR (1AT21CS085), ROHITH H CHAVAN (1AT21CS086), SANSKAR RAJ (1AT21CS097), MUTHURAJ R (1AT22CS407), students of VI semester, B.E in Computer Science & Engineering at **Atria Institute of Technology**, hereby declare that the project work entitled "**DIGITAL IMAGE FORGERY DETECTION**" has been carried out under the supervision of Prof. Keerthi KS, Professor, Dept. of CS&E, Atria Institute of Technology and submitted in partial fulfilment of the course requirements for the award of degree in B.E in Computer Science & Engineering of Visvesvaraya Technological University, Belagavi during the year 2023-2024. I further declare that the report has not been submitted to any other University for the award of any other degree.

Place: Bangalore
Date:

Signature of Students

RISHABH KUMAR
(1AT21CS085)

ROHITH H CHAVAN
(1AT21CS086)

SANSKAR RAJ
(1AT21CS097)

MUTHURAJ
(1AT22CS407)

ABSTRACT

The OpenGL project that demonstrates the bubble sorting algorithm. The project comprises a user-friendly menu page with options to start or view credits. Clicking on the credits page will display the USN and names of the team members. Upon selecting start, the user will be directed to input numbers through a graphics interface. Once the numbers are entered, the project will graphically represent the array and sort it using the bubble sorting algorithm in real-time. The interface will be engaging with bright colors, animations, and clear instructions to make learning about the algorithm an enjoyable experience. The project aims to provide a valuable educational tool for students and programming enthusiasts interested in learning about algorithms.

ACKNOWLEDGEMENT

I express my gratitude to our institution and management for providing me with good infrastructure, laboratory, facilities and inspiring staff, and whose gratitude was of immense help in completion of this project successfully.

I express my sincere gratitude to **Dr. RAJESHA S,** Principal, Atria Institute of Technology, for providing me the required environment and for her valuable suggestion.

My sincere thanks to **Dr. DEVI KANNAN,** Head of the Dept. Computer Science and Engineering, Atria Institute of Technology, for her valuable support and for rendering me resources for this project work.

I express my gratitude to **Prof. KEERTHI KS,** Assistant Professor, Dept. of Computer Science and Engineering, Atria Institute of Technology, who guided me with valuable suggestions in completing this project at every stage.

Last but not the least, the project would not have been a success without the support of my **parents** and **friends**. My sincere thanks should be rendered to everyone who helped me in all possible ways

TABLE OF CONTENTS

DECLARATION i

ABSTRACT ii

ACKNOWLEDGMENT iii

CHAPTER I INTRODUCTION .....[1-2]

CHAPTER II BACKEND OF PROJECT ..... [3-6]

CHAPTER III WORKING OF PROJECT..... [7-12]

CHAPTER IV SYSTEM DESIGN.....[13-16]

CONCLUSION .....[17]

FUTURE ENHANCEMENT ..... [18]

REFERENCES..... [19]

CHAPTER I

INTRODUCTION

In today's digital age, the manipulation and alteration of digital images have become increasingly prevalent and sophisticated. Digital image forgery, often performed with malicious intent, can range from subtle alterations to complete fabrications, posing significant challenges to the authenticity andcredibility of visual information. As such, the ability to detect and mitigate such forgeries has becomea critical area of research and development.

The proliferation of image editing software, coupled with the ease of sharing images across various platforms, has facilitated the rapid spread of manipulated images. These forgeries can be used for diverse purposes, including spreading misinformation, manipulating public opinion, or falsifying evidence in legal or forensic contexts. Detecting such manipulations is crucial not only for maintaining the integrity of digital content but also for upholding trust and accountability in media and communication.

Advancements in image editing software have democratized the ability to manipulate images, enabling users to alter photographs with unprecedented ease and sophistication. While these tools have empowered creativity and artistic expression, they have also opened avenues for malicious actors to create and disseminate misleading or falsified images. Such forgeries can range from subtletouch-ups to sophisticated composites that convincingly alter reality.

The consequences of digital image forgery can be profound. In journalism and media, manipulated images can distort public perception, misrepresent events, or propagate false narratives. In legal andforensic contexts, forged images can undermine the validity of evidence, potentially affecting judicialoutcomes. Moreover, in fields such as medical imaging and scientific research, the authenticity of images is crucial for accurate diagnosis, analysis, and scholarly integrity.

Types of Digital Image Forgery

Copy-Move Forgery: Involves duplicating one or more regions of an image and pasting them within the same image to create the illusion of additional content or to conceal unwanted elements

Splicing: Combines parts of multiple images to create a new composite image, often used to fabricate events, mislead viewers, or alter historical documentation.

Object Removal/Addition: Involves altering an image by either removing existing objects or adding new ones, thereby changing the scene's narrative or visual context.

Approaches to Detection

Forensic Analysis: Examines metadata embedded within images, such as timestamps and camerasettings, to identify inconsistencies or anomalies that suggest tampering.

Pixel-Based Analysis: Analyzes the spatial and color distribution of pixels to detect abrupt changes or inconsistencies that may indicate regions of an image that have been manipulated.

Frequency Domain Analysis: Utilizes Fourier transform and similar techniques to analyze frequency components within images, identifying discrepancies that may arise from splicing or otherforms of manipulation.

Machine Learning and AI: Employs supervised and unsupervised machine learning algorithms, including convolutional neural networks (CNNs), to automatically learn and distinguish between authentic and manipulated images based on learned patterns and features.

CHAPTER 2

BACKEND OF PROJECT

```
import tkinter as tk from tkinter.filedialog import
askopenfilename from tkinter import messagebox
import osimport cv2 from numpy import result_type
from signature import match

# Mach Threshold
THRESHOLD = 85


def browsefunc(ent):
filename =
askopenfilename(filetypes=([
("image", ".jpeg"),
("image", ".png"),
("image", ".jpg"),
   ]))                    ent.delete(0,
tk.END)                   ent.insert(tk.END,
filename)  # add this


def capture_image_from_cam_into_temp(sign=1):
                                          cam =
cv2.VideoCapture(0, cv2.CAP_DSHOW)
cv2.namedWindow("test")

#
img_counter
= 0while
True:
      ret, frame = cam.read()  if not
ret:print("failed to grab frame")
                            break
cv2.imshow("test", frame)

      k = cv2.waitKey(1)
            if k % 256 == 27:                  #
ESC pressed print("Escape hit, closing...")
break                           elif k % 256 ==
32: # SPACE pressedif not os.path.isdir('temp'):
            os.mkdir('temp', mode=0o777)  # make sure the directory exists  # img_name =
"./temp/opencv_frame_{}.png".format(img_counter) if(sign == 1):
img_name = "./temp/test_img1.png"           else:
            img_name = "./temp/test_img2.png"  print('imwrite=',
cv2.imwrite(filename=img_name, img=frame))     print("{}
written!".format(img_name))
```

```
    #'SUCCESS!!!', 'Press Space Bar to click picture and ESC to exit')
    res = None
    res =messagebox.askquestion(
'Click Picture', 'Press Space Bar to click picture and ESC to exit')  if res == 'yes':
capture_image_from_cam_into_temp(sign=sign)
ent.delete(0, tk.END)
ent.insert(tk.END, filename)
return True

def checkSimilarity(window, path1, path2):            result =
match(path1=path1, path2=path2)       if(result <= THRESHOLD):
messagebox.showerror("Failure: Images Do Not Match",
                     "Images are "+str(result)+f" % similar!!")     pass     else:
messagebox.showinfo("Success: Images Match",
" Images are "+str(result)+f" % similar!!")                    return True


root = tk.Tk() root.title("Signature
Matching")root.geometry("500x700")
# 300x200
uname_label = tk.Label(root, text="Compare Two Images:", font=10) uname_label.place(x=90,
y=50)img1_message = tk.Label(root, text="Image 1", font=10)
img1_message.place(x=10, y=120)

image1_path_entry = tk.Entry(root,
font=10)
image1_path_entry.place(x=150,
y=120)
 img1_capture_button = tk.Button(            root,
text="Capture", font=10,command=lambda:
captureImage(ent=image1_path_entry, sign=1))
img1_capture_button.place(x=400, y=90)
img1_browse_button = tk.Button(root,
text="Browse", font=10, command=lambda:
browsefunc(ent=image1_path_entry))
img1_browse_button.place(x=400, y=140)
image2_path_entry = tk.Entry(root, font=10)
image2_path_entry.place(x=150, y=240)
 img2_message = tk.Label(root, text="Image 2", font=10)img2_message.place(x=10, y=250)
 img2_capture_button = tk.Button(            root,
text="Capture", font=10,command=lambda:
img2_browse_button = tk.Button(root,
text="Browse", font=10, command=lambda:
browsefunc(ent=image2_path_entry))
img2_browse_button.place(x=400, y=260)
 compare_button = tk.Button(                                        root, text="Compare", font=10,
command=lambda: checkSimilarity(window=root,
 path1=image1_pat h_entry.get(),path2=image2_pat h_entry.get(),))
```

1. Import Necessary Libraries

The script starts by importing libraries for data handling, image processing, machine learning modelbuilding, and visualization. Key libraries include tensorflow, keras, numpy, pandas, and matplotlib.

2. Check and Load Dataset

The script verifies the existence of the dataset directory. It then iterates through the directorystructure to print out file paths, ensuring that the dataset is properly loaded.

3. Retrieve Labels

The labels for the fruit classes are extracted from the subdirectory names within the training datasetdirectory.

4. Data Preprocessing

The ImageDataGenerator class from Keras is used to rescale the image pixel values and split the dataset into training and validation sets. This helps to augment the data and enhance the model's ability to generalize.

5. Build the CNN Model

The CNN model is built using the Keras Sequential API. The architecture consists of convolutional layers for feature extraction, max-pooling layers for down-sampling, a flatten layerto convert 2D matrices to 1D, and dense layers for classification.

6. Compile the Model

The model is compiled with the Adam optimizer, categorical cross-entropy loss function, and accuracy as the performance metric.

7. Visualize Training Progress

The script plots the training and validation accuracy and loss over epochs to visualize the model's performance and detect any signs of overfitting The trained model is tested on a separate test set toevaluate its performance on unseen data. Predictions are made using the test data generator.

8. Save the Model

Finally, the trained model is saved to a file for future use or deployme

CHAPTER 3

FRONT END OF PROJECT

This HTML document creates a user interface for displaying the results of a fruit freshnessprediction. Here's a detailed summary of the front-end components and their functions:

```
import tkinter as tk from tkinter.filedialog import askopenfilename from tkinter import
messagebox import osimport cv2 from numpy import result_type from signature import match

# Mach Threshold THRESHOLD = 85


def browsefunc(ent):
filename = askopenfilename(filetypes=([("image", ".jpeg"),
("image", ".png"),
("image", ".jpg"),
   ]))                          ent.delete(0, tk.END)       ent.insert(tk.END,filename)  # add this


def capture_image_from_cam_into_temp(sign=1): cam =cv2.VideoCapture(0,
cv2.CAP_DSHOW)

cv2.namedWindow("test")# img_counter = 0
while True:
      ret, frame = cam.read()  if not ret:print("failed to grab frame")              break
cv2.imshow("test", frame)

      k = cv2.waitKey(1)  if k %256 == 27:
         # ESC pressed print("Escape hit, closing...")           break     elif k % 256 == 32:
# SPACE pressed if not os.path.isdir('temp'):
            os.mkdir('temp', mode=0o777)  # make sure the directory exists  # img_name =
"./temp/opencv_frame_{}.png".format(img_counter) if(sign == 1):
img_name = "./temp/test_img1.png"               else:
```

```
img_name = "./temp/test_img2.png"
print('imwrite=', cv2.imwrite(filename=img_name, img=frame))
print("{} written!".format(img_name))
# img_counter += 1
cam.release()cv2.destroyAllWindows()
return True


def captureImage(ent, sign=1):     if(sign == 1):
filename = os.getcwd()+'\\temp\\test_img1.png' else:
 filename = os.getcwd()+'\\temp\\test_img2.png'# messagebox.showinfo(
#'SUCCESS!!!', 'Press Space Bar to click picture and ESC to exit')
res = None
res =messagebox.askquestion(
'Click Picture', 'Press Space Bar to click picture and ESC to exit')
if res == 'yes':capture_image_from_cam_into_temp(sign=sign)
ent.delete(0, tk.END)
ent.insert(tk.END, filename)
return True


def checkSimilarity(window, path1, path2):
result = match(path1=path1, path2=path2) if(result <= THRESHOLD):
messagebox.showerror("Failure: Images Do Not Match",
                     "Images are "+str(result)+f" % similar!!")     pass     else:
messagebox.showinfo("Success: Images Match",
" Images are "+str(result)+f" % similar!!")
return True

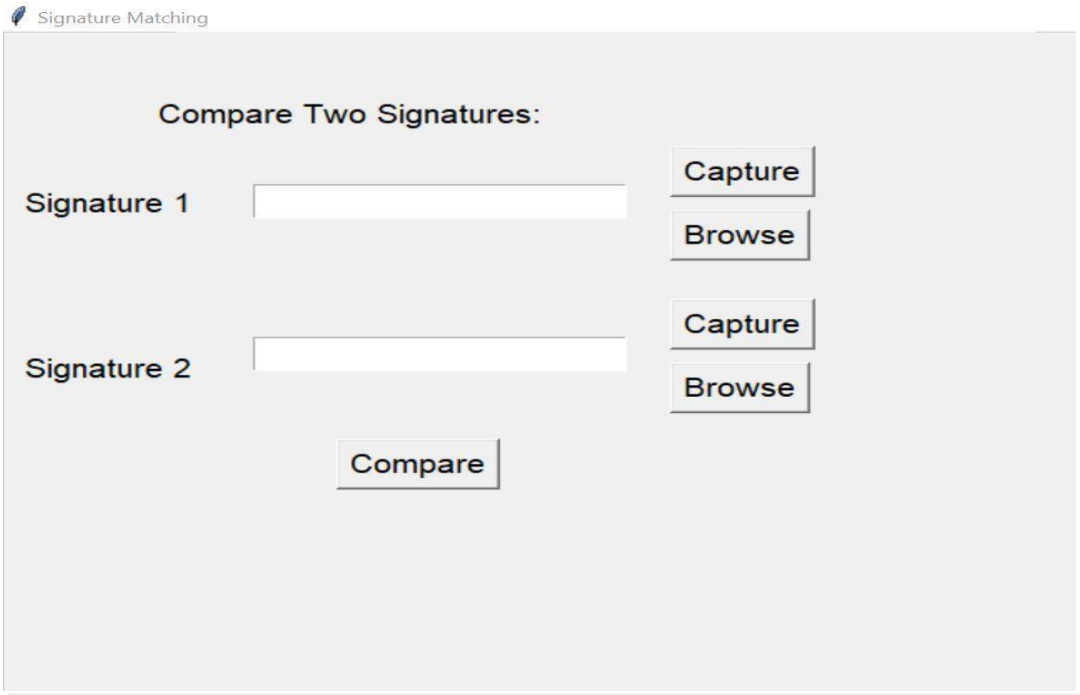

root = tk.Tk() root.title("Signature Matching")
root.geometry("500x700") # 300x200 uname_label = tk.Label
(root, text="Compare Two Images:", font=10) uname_label.place(x=90, y=50)
 img1_message = tk.Label(root, text="Image 1", font=10)
 img1_message.place(x=10, y=120)
 image1_path_entry = tk.Entry(root, font=10)
 image1_path_entry.place(x=150, y=120)
 img1_capture_button = tk.Button(            root, text="Capture", font=10,command=lambda:
captureImage(ent=image1_path_entry, sign=1))
img1_capture_button.place(x=400, y=90)
img1_browse_button = tk.Button(root, text="Browse", font=10,
command=lambda: browsefunc(ent=image1_path_entry))
img1_browse_button.place(x=400, y=140) image2_path_entry = tk.Entry(root, font=10)
image2_path_entry.place(x=150, y=240)
 img2_message = tk.Label(root, text="Image 2", font=10)
 img2_message.place(x=10, y=250)
 img2_capture_button = tk.Button(
 root, text="Capture", font=10,command=lambda:
captureImage(ent=image2_path_entry, sign=2))
 img2_capture_button.place(x=400, y=210)
img2_browse_button = tk.Button(root, text="Browse", font=10, command=lambda:
```

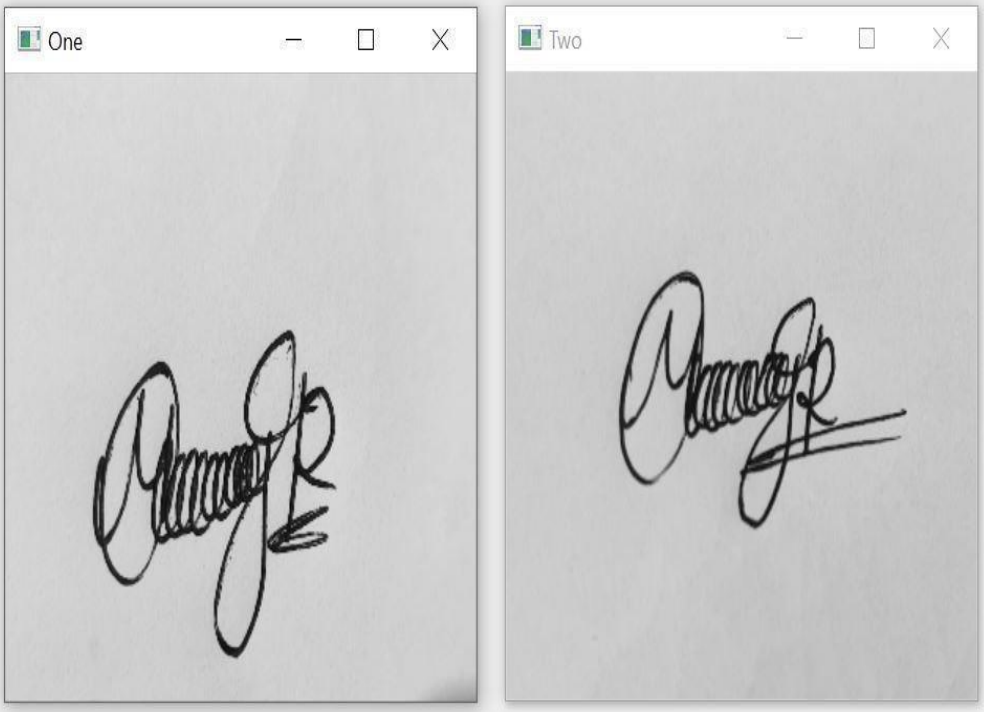
```
browsefunc(ent=image2_path_entr
y))
img2_browse_button.place(x=400,
y=260)compare_button =
tk.Button(
   root, text="Compare", font=10, command=lambda:
checkSimilarity(window=root,path1=image1_pat h_entry.get(),
                                                path2=image2_pat

h_entry.get(),))compare_button.place(x=200, y=320)

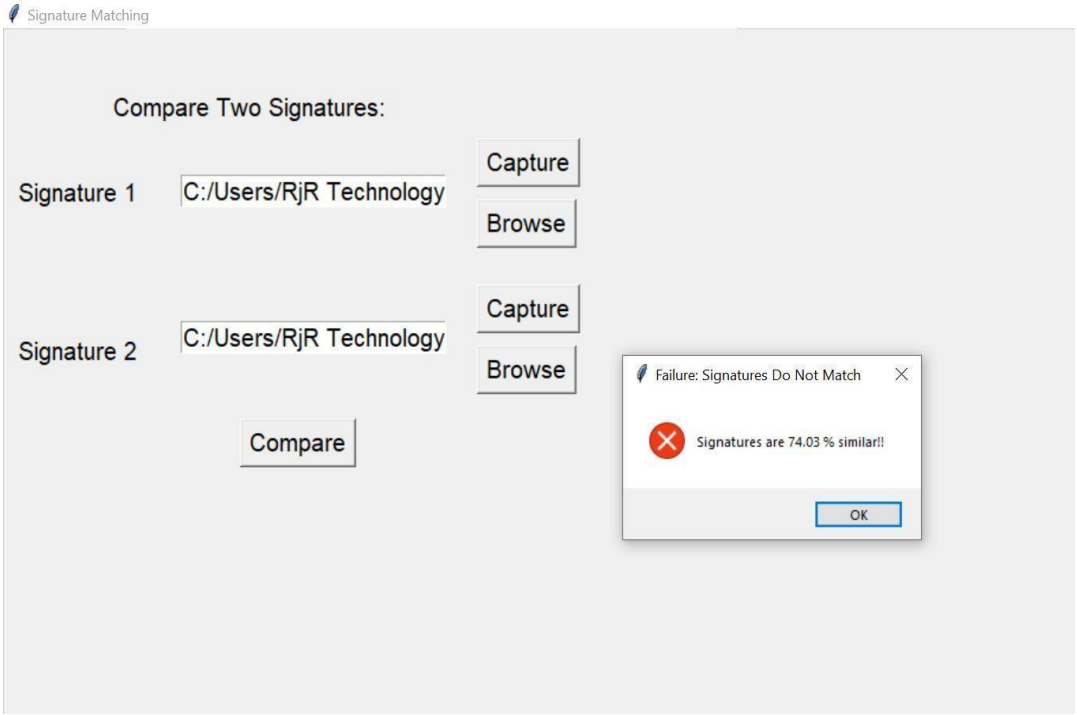
root.mainloop()
```

Output

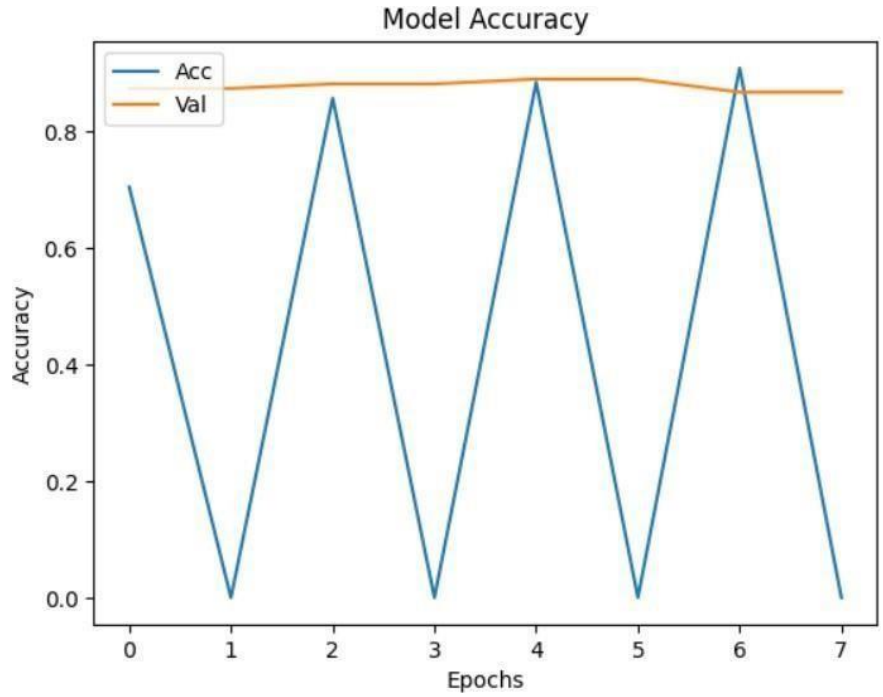

Signature Matching
Compare Two Signatures:
Signature 1
Capture
Browse
Signature 2
Capture
Browse
Compare

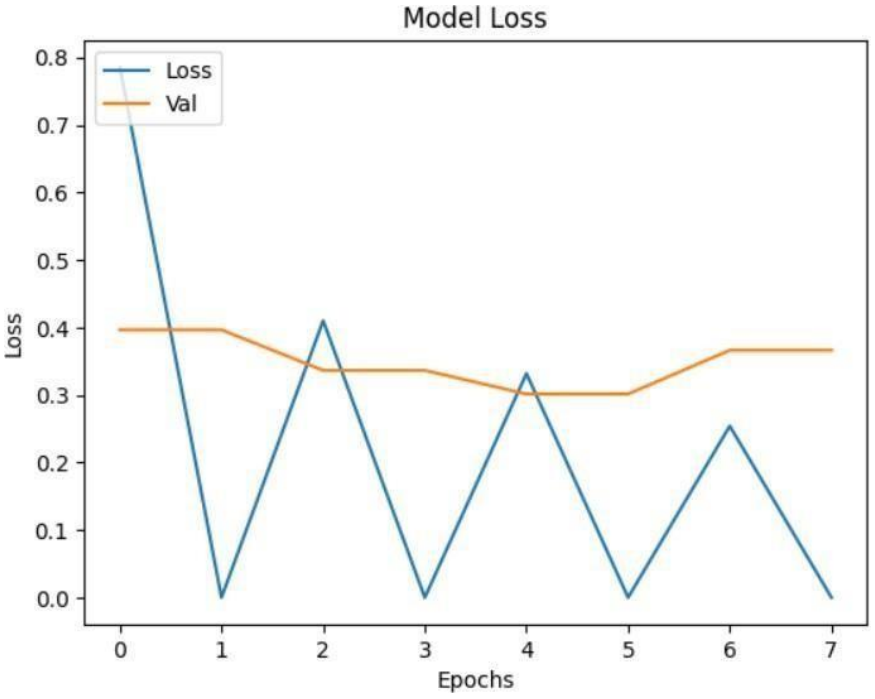

One
Two


Failure: Signatures Do Not Match
Signatures are 74.03 % similar!!
OK


Signature Matching
Compare Two Signatures:
Signature 1
C:/Users/RjR Technology
Capture
Browse
Signature 2
C:/Users/RjR Technology
Capture
Browse
Compare
Failure: Signatures Do Not Match
Signatures are 74.03 % similar!!
OK

Model-graph:


Model Accuracy
Acc
Val
Accuracy
Epochs
0.0
0.2
0.4
0.6
0.8
0
1
2
3
4
5
6
7

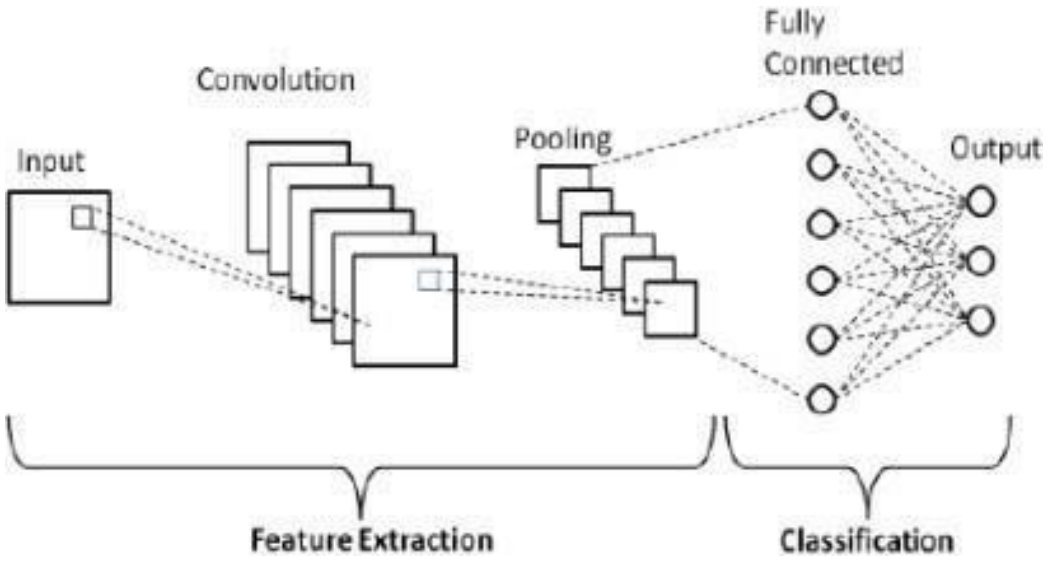

Model Loss
Loss
Val
Loss
Epochs
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
1
2
3
4
5
6
7

CHAPTER 4

SYSTEM DESIGN

The system design section includes the Flowchart and its explanation.

FLOW CHART:



1. HTML/CSS/JavaScript:

- HTML: Structure of the web pages (upload page, result page).
- CSS: Styling for the layout, container, text, images, and buttons.
- JavaScript: Handle file uploads, make asynchronous requests to the server, and dynamicallyupdate the UI.

2. Web Server (Back-End)Flask (Python):

- File Upload Endpoint: Accepts image uploads from the front-end.
- Prediction Endpoint: Processes the uploaded image, invokes the ML model, and returns theprediction result.
- Serve Static Files: Serves the HTML, CSS, and JavaScript files to the client.

3. Machine Learning Model TensorFlow/Keras (Python):

- Pre-trained Model: A convolutional neural network (CNN) trained to classify images offruit as either fresh or rotten.
- Prediction Function: Takes preprocessed images as input and outputs the prediction result.

4. Image StorageFile System:

- Temporary Storage: Stores uploaded images temporarily until they are processed.

5. Data Flow

1. User Interaction:

- User navigates to the web application.
- User uploads an image of fruit through the front-end form.

2. Image Upload:

- The front-end sends the uploaded image to the Flask back-end via an HTTP POSTrequest.

3. Image Preprocessing:

- The Flask back-end preprocesses the image (resizing, normalization) to the required inputformat for the ML model.

4. Prediction:

- The preprocessed image is passed to the TensorFlow/Keras model
- The model outputs a prediction (mtching or not).

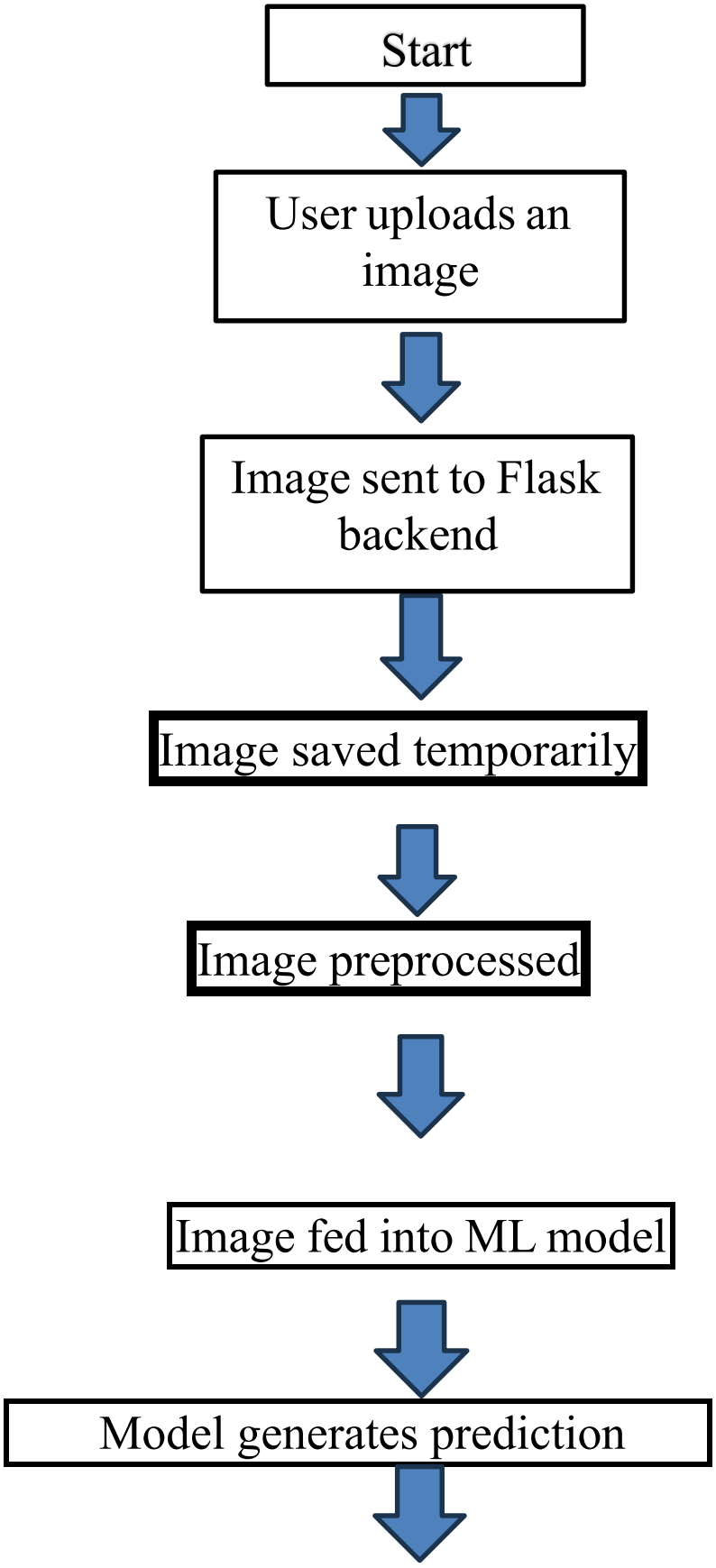
5. Display Result:

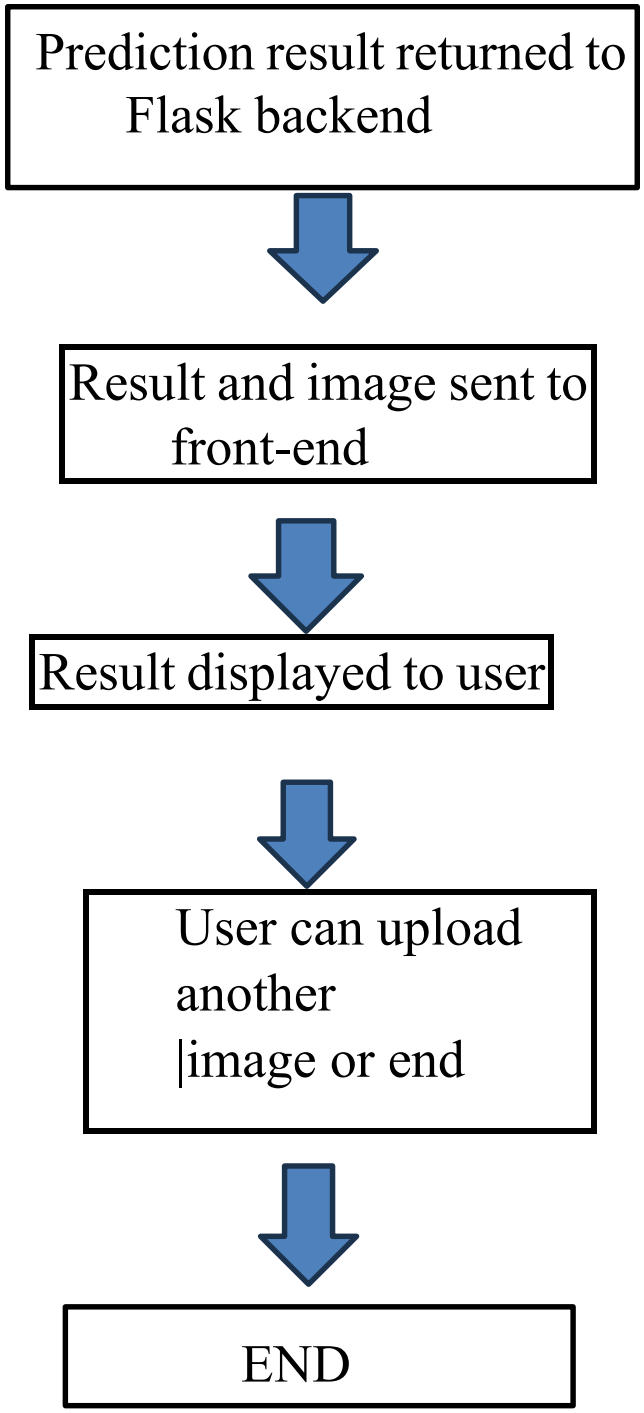
- The prediction result and the uploaded image are sent back to the front-end.
- The front-end displays the result to the user, showing whether the two images are matchingor not.

6. User Action:

- The user can choose to upload another image or end the session

FLOW DIAGRAM :




Prediction result returned to Flask backend
Result and image sent to front-end
Result displayed to user
User can upload another |image or end
END

CONCLUSION

The fruit freshness detection project demonstrates a cohesive integration of front-end web development, back-end server processing, and machine learning model implementation. Here's a concise conclusion summarizing the project:

Achievement of Objectives

The project successfully achieves its primary objectives:

- User Interface: Provides a user-friendly interface where users can upload images of fruits to determine their freshness.
- Back-End Processing: Utilizes Flask to handle image uploads, preprocess images, invoke a pre-trained TensorFlow/Keras model for prediction, and return results to the front-end.
- Machine Learning Model: Implements a convolutional neural network (CNN) trained to classify fruit images as either fresh or rotten, ensuring accurate predictions based on the image data.
- Integration and Flow: Integrates seamlessly from user interaction through image processing to result display, ensuring a smooth and intuitive experience.

Key Features and Benefits

- User Interaction: Enables users to easily upload images via a web interface, enhancing accessibility and usability.
- Real-Time Prediction: Provides instant feedback on the freshness of fruits, leveraging machine learning to automate and enhance decision-making.
- Scalability and Maintainability: Designed with modularity and scalability in mind, facilitating future enhancements and maintenance.
- Cross-Platform Compatibility: Supports various operating systems and browsers, ensuring accessibility for a wide range of users.

FUTURE ENHANCEMENT

Digital image forgery detection is a crucial task in various fields like law enforcement, media, and security. Enhancing digital image forgery detection involves improving the accuracy and efficiency of algorithms to identify tampered images. Here are some ways to enhance digital image forgery detection:

1. Deep Learning Techniques: Utilize deep learning-based methods like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to analyze image patterns and detect anomalies.

2. Image Segmentation: Implement image segmentation techniques to divide images into regions and analyze each region separately for tampering.

3. Feature Extraction: Extract robust features from images, such as texture, color, and shape, to identify inconsistencies.

4. Anomaly Detection: Employ anomaly detection algorithms to identify unusual patterns or features in images.

5. Multi-Scale Analysis: Analyze images at multiple scales to detect tampering in different regions.

6. Fusion of Multiple Methods: Combine multiple detection methods to improve accuracy.

7. Large-Scale Datasets: Train algorithms on large-scale datasets to enhance performance.

8. Regular Updates: Regularly update algorithms to adapt to new tampering techniques.

REFERENCES

- Basics:https:///www.geeksgorgeeks.org/digital-image-0processing- basics
- https://www.wikipedia.org/
- https://www.w3schools.com/
- https://github.com/
- TensorFlow: https://www.tensorflow.org